

Interpolation

CL249 25-08-2026

Newton's Divided Differences

Divided Differences Concept

Base & First-Order Definitions

0th order divided difference is the function value itself:

$$f[x_i] = f(x_i)$$

1st order divided difference represents the discrete average slope:

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

2nd order divided difference represents the discrete average slope:

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+2}, x_{i+1}] - f[x_{i+1}, x_i]}{x_{i+2} - x_i}$$

General Recursive Relation

Higher order divided differences are constructed recursively from lower orders:

$$\begin{aligned} f[x_i, \dots, x_{i+k}] \\ = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i} \end{aligned}$$

Invariance property: The order of arguments inside $f[\cdot]$ does not affect the output value.

Newton's Interpolation Algorithm

Given: Data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$.

Step 1: Initialize divided difference table with:

$$f[x_i] = y_i, \quad i = 0, 1, 2, \dots, n$$

Step 2: Calculate divided differences for orders 1 through n using the

formula:
$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

For each increasing order $k = 1, 2, \dots, n$, compute these for all valid i .

Step 3: Construct the Newton's interpolating polynomial:

$$\begin{aligned} P(x) &= f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots \\ &+ (x - x_0)(x - x_1) \cdots (x - x_{n-1})f[x_0, x_1, \dots, x_n] \end{aligned}$$

Step 4: To evaluate the polynomial at any specific value $x = a$, compute:

- Initialize $P(a) = f[x_0]$
- Initialize the product term as 1
- For each term $j = 1$ to n :
 - Update product term: multiply by $(a - x_{j-1})$
 - Add to $P(a)$ the product term times $f[x_0, x_1, \dots, x_j]$

Step 5: Output $P(a)$ as the interpolated value at $x = a$.

Divided Difference Table Structure

Node (x_i)	$f(x_i)$ [0th DD]	1st Divided Difference	2nd Divided Difference	3rd Divided Difference
x0	$f[x_0] \rightarrow (a_0)$	$f[x_0, x_1] \rightarrow (a_1)$	$f[x_0, x_1, x_2] \rightarrow (a_2)$	$f[x_0, x_1, x_2, x_3] \rightarrow (a_3)$
x1	$f[x_1]$	$f[x_1, x_2]$	$f[x_1, x_2, x_3]$	-
x2	$f[x_2]$	$f[x_2, x_3]$	-	-
x3	$f[x_3]$	-	-	-

Note: The top diagonal entries (highlighted with arrows) directly form coefficients a_k in Newton's expansion polynomial.

Newton's Interpolating Polynomial

Triangular Basis Expansion

Newton's form expresses $P_n(x)$ in a nested basis structure:

$$P_n(x) = f[x_0] + \sum_{k=1}^n (f[x_0, x_1, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j))$$

$O(n)$

Incremental Node
Addition

Incremental Advantage: Adding point x_{n+1} adds only one new term without modifying any previously computed coefficients!

Local Newton's Divided Differences

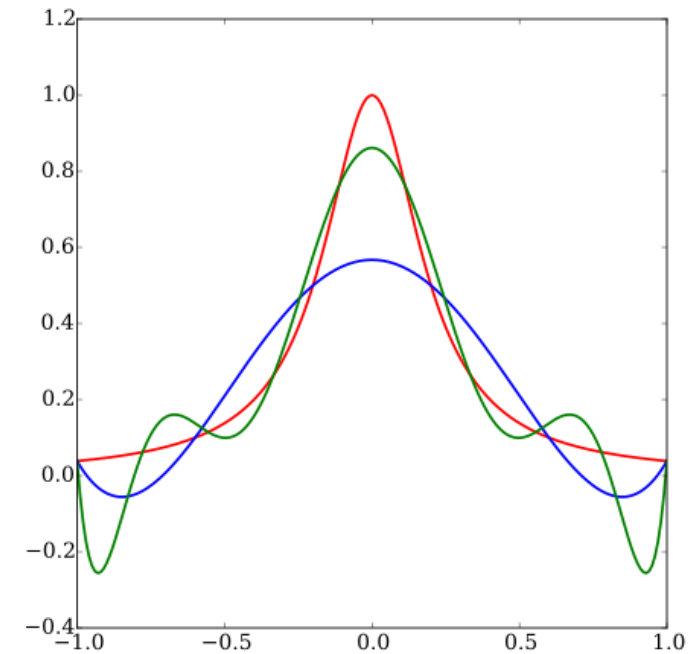
Piecewise local stencils to eliminate global Runge oscillations and ensure stability

Local Newton Formulations

Mitigating Runge's Phenomenon

High-degree global polynomial interpolation suffers from severe edge oscillations when using uniform node grids.

Local Newton interpolation selects a compact moving neighborhood of k nearest points around target evaluation point x . It computes localized low-degree divided differences, preserving sharp gradients while preventing numerical runaway.



Local Newton Algorithm Workflow

1. Stencil Selection

For target point x , find index i such that $x_i \leq x \leq x_{i+k}$. Select $k + 1$ nearest neighboring nodes $\{x_{i0}, \dots, x_{ik}\}$ around x .

2. Local DD Table

Compute local divided differences for the small $(k + 1)$ stencil up to degree k (typically $k = 2$ or 3):

3. Horner Evaluation

Evaluate local interpolant $P_k(x)$ efficiently using nested multiplication:

$$\begin{aligned} P_k(x) &= a_0 + (x - x_0)a_1 + (x - x_0(x - x_1))a_2 + \dots \end{aligned}$$

4. Adaptive/ENO Check

In ENO/WENO schemes, compare divided difference magnitudes to shift stencil away from discontinuities and shocks.

Lagrange Polynomials

Constructing global interpolating polynomials
using explicit orthogonal basis functions

Theoretical Foundations

The Interpolation Problem

Given a set of **$n + 1$** distinct sample

points: $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$

Find a polynomial **$P(x)$** of degree at most **n** such that it passes exactly through all points: $P(x_i) = y_i$ for all **$i = 0, \dots, n$** .

Lagrange Basis Formulation

Basis Functions

Define Lagrange basis polynomials $L_i(x)$ of degree n for each node:

$$L_i(x) = \prod_{j=0, j \neq i}^n \left(\frac{x - x_j}{x_i - x_j} \right)$$

Delta Property

Each basis polynomial satisfies the Kronecker delta property at node locations:

$$L_i(x_k) = \delta_{ik}$$

Total Interpolant

The explicit linear combination yields the complete interpolating polynomial:

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

Lagrange Properties & Drawbacks

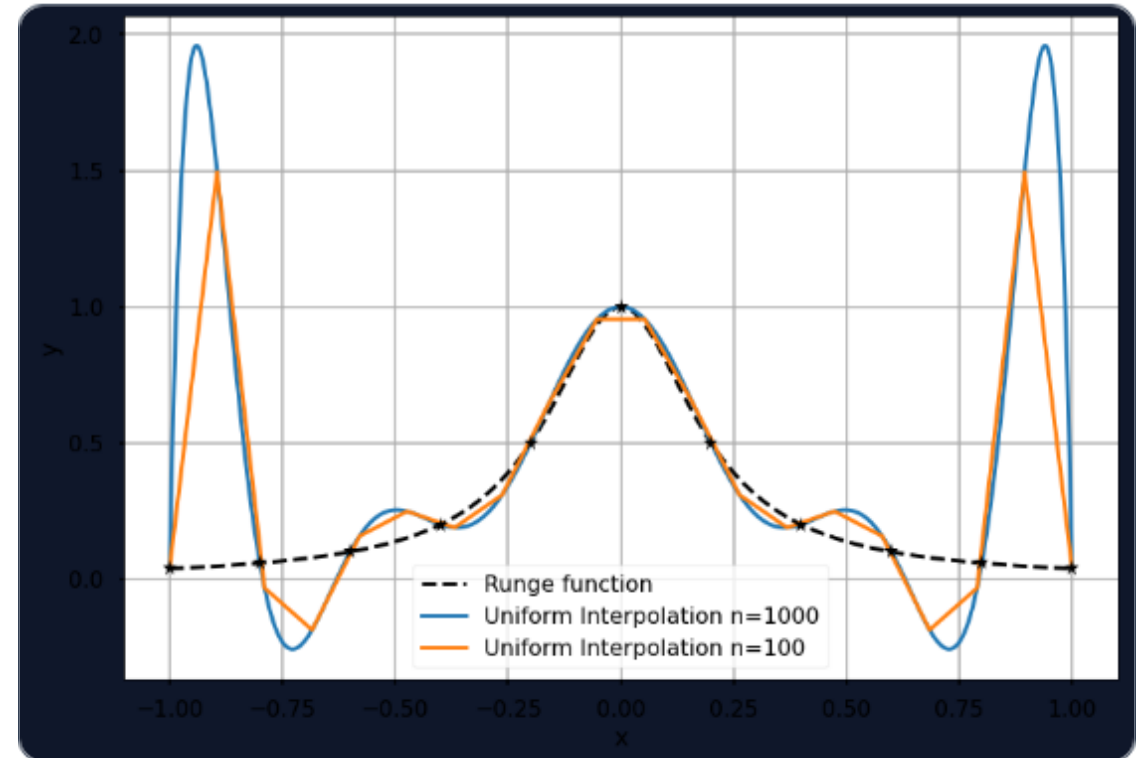
Key Operational Characteristics

Explicit & Direct: Requires no linear system solver or matrix inversion, making theoretical derivations straightforward.

High Computational Cost: Evaluating $P(\mathbf{x})$ at a single point requires $O(n^2)$ arithmetic operations.

Lack of Extensibility: Adding a new point (x_{n+1}, y_{n+1}) invalidates all previous basis functions $L_i(x)$, requiring full recalculation from scratch.

$$f(x) = 1 + \frac{1}{1 + 25x^2}$$



Algorithm for Lagrange Interpolation

Given:

A set of $n+1$ data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ where all x_i are distinct.

Step 1: For each point i , construct the Lagrange basis polynomial $L_i(x)$:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

This polynomial satisfies $L_i(x_i) = 1$ and $L_i(x_j) = 0$ for $j \neq i$.

Step 2: Form the interpolating polynomial $P(x)$ as the weighted sum:

$$P(x) = \sum_{i=0}^n y_i \cdot L_i(x)$$

Step 3: To interpolate at any desired value $x = a$, evaluate $P(a)$ by computing each $L_i(a)$ and summing:

For each i from 0 to n :

- Calculate $L_i(a)$ using the product formula in Step 1.

Compute $P(a) = \sum_{i=0}^n y_i \times L_i(a)$.

Note: The polynomial $P(x)$ passes exactly through all given points and has degree at most n .

Comprehensive Method Comparison

Feature	Lagrange Form	Global Newton DD	Local Newton DD
Construction Cost	$O(n^2)$ operations	$O(n^2)$ table fill	$O(k^2)$, $k \ll n$ local stencil
Point Addition	Must recalculate all basis	$O(n)$ append new row	$O(k)$ localized update
Runge Oscillation	Severe for large n	Severe for large n	Eliminated by localized bounds
Primary Application	Theoretical proofs & quadrature	Table lookup & derivative estimation	ENO/WENO schemes & spline fitting